

***Universitatea Ovidius din Constanta
Facultatea de Matematica si Informatica
Specializarea Informatica***

Ingineria sistemelor software

Curs 1

Conf. univ. dr. Crenguta M. Puchianu
2020

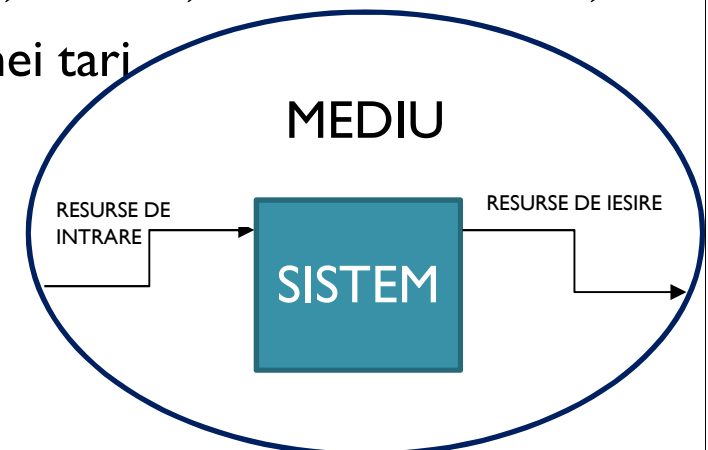


Partea I. Introducere



Sisteme

- Sistem = colectie de componente ce interactioneaza intre ele intr-un mod unitar
- Sisteme:
 - Naturale: univers, atom, galaxie, ecosistem, sistemul anatomic, etc
 - Artificiale: masini, avioane, sisteme de control al zbourilor, sisteme bancare, calculatoare, etc
- Resurse: materiale, energetice si informationale
- Orice sistem are o stare si un comportament
- Dpdv al comportamentului, sistemele se impart in:
 - Simple: biciclete, etc
 - Complexe si organizate: masini, vapoare, avioane, sisteme hardware, etc
 - Complexe si neorganizate: economia unei tari, bursa marfurilor, etc



Sistem informatic si sistem software

- Sistem informatic = sistem complex si organizat format din:
 - Sistem software
 - Sistem hardware
 - Oameni
 - Baze date si/sau de cunostinte
 - Documentatie
- Sistem software = sistem complex si organizat format din:
 - Programe
 - Structuri de date
 - Documentatie
- Caracteristicile sistemelor software:
 - Nu sunt fabricate in sens clasic, ci dezvoltate sau inginerizate
 - Nu se demodeaza, dar se pot deteriora
 - Majoritatea sunt create de la zero, nu din compunerea componentelor existente



Calitatea sistemelor software

Criteriu	Definitie
Timp de raspuns	Cat de repede stie sistemul de cererea utilizatorului dupa ce aceasta a fost transmisa?
Iesirea	Cat de multe task-uri realizeaza sistemul intr-o perioada de timp?
Memoria	De cata memorie are nevoie sistemul pentru a rula?
Robustete	Abilitatea de a trata o intrare invalida de la utilizator
Siguranta	Diferenta dintre comportamentul specificat si cel observat
Valabilitate	Procentul din timpul sistemului folosit pentru realizarea sarcinilor normale
Toleranta implicita	Abilitatea de a opera in conditii eronate
Securitate	Abilitatea de a raspunde la fortari a sistemului
Protectie	Abilitatea de a nu pune in pericol vietii omenesti, chiar in prezenta erorilor si esecurilor
Costul de dezvoltare	Costul dezvoltarii sistemului
Costul de repartitie	Costul instalarii sistemului si pregatirea utilizatorilor
Costul upgrad-arii	Costul translatarei datelor de la sistemul anterior, in cazul reengineering. Acest criteriu rezulta din cerintele de compatibilitate cu sistemul initial.
Costul de intretinere	Costul necesar rezolvarii bug-urilor si imbunatatirii sistemului
Costul de administrare	Banii necesari administrarii sistemului



Calitatea sistemelor software (cont.)

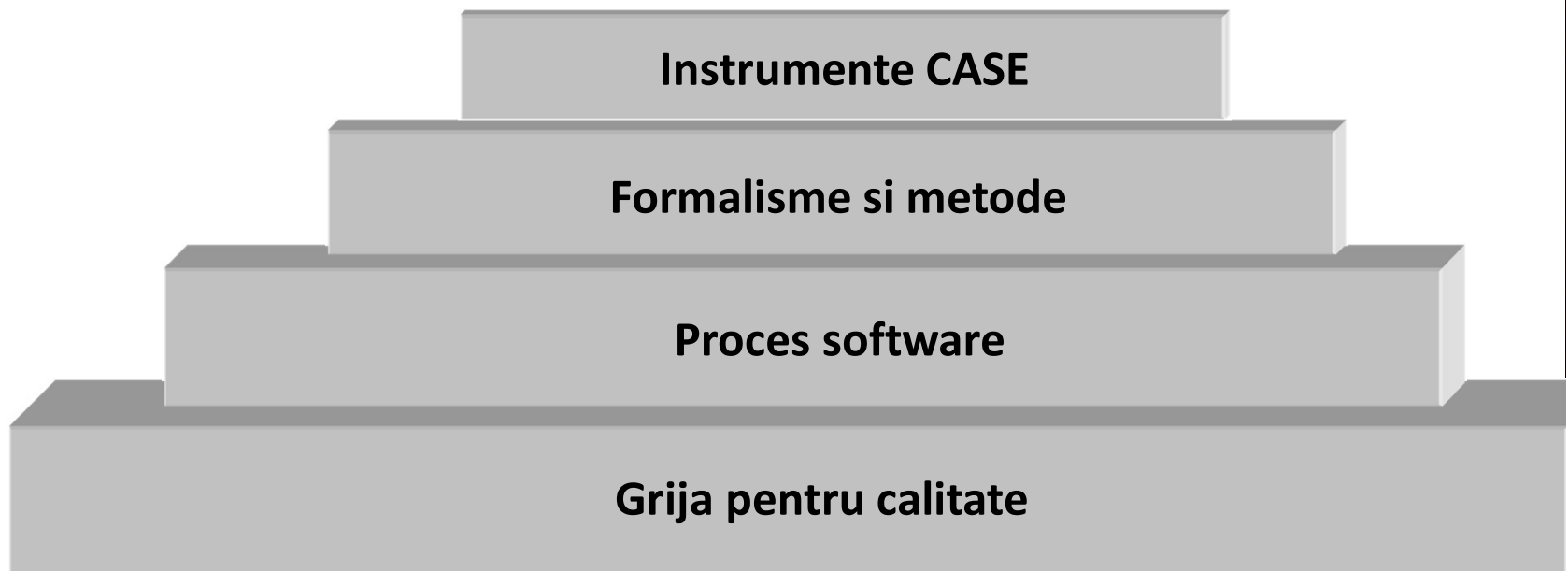
Extensibilitate	Cat de usor este sa adaugam noi functionalitati sau noi clase la sistem?
Modificabilitate	Cat de usor modificam functionalitatea sistemului?
Adaptabilitate	Cat de usor portam sistemul la un nou domeniu al problemei?
Portabilitate	Cat de usor portam sistemul la o platforma diferita?
Lizibilitate	Cat de usor se intelege sistemul citind codul acestuia?
Urmărirea cerintelor	Cat de usor mapam codul la cerintele sistemului?
Utilitate	Cat de bine suporta sistemul munca utilizatorului?
Utilizare	Cat ii este de usor utilizatorului sa foloseasca sistemul?

COMPROMIS	Motivatia
Spatiu vs viteza	Daca software-ul nu indeplineste timpul de raspuns impus sau cerintele de iesire, poate fi expandata mai multa memorie pentru a mari viteza software-ului.
Termenul de livrare vs functionalitate	Daca dezvoltarea a depasit orarul, manager-ul proiectului poate livra la timp produsul dar cu mai putina functionalitate de cat era planificata initial, sau sa furnizeze cu toata functionalitatea specificata dar depasind termenul
Termen de livrare vs calitate	Daca testarea a depasit orarul, managerul proiectului poate livra sw-ul la timp, dar cu bug-uri stiute (si apoi sa le rezolve) sau sa livreze mai tarziu, dar cu cat mai multe bug-uri rezolvate
Termen de livrare vs staff	Daca dezvoltarea a depasit orarul, managerul proiectului poate adauga resurse pentru a creste productivitatea, dar care determina cresterea costului de dezvoltare. Pentru majoritatea proiectelor, aceasta optiune este valabila la inceputul proiectului.



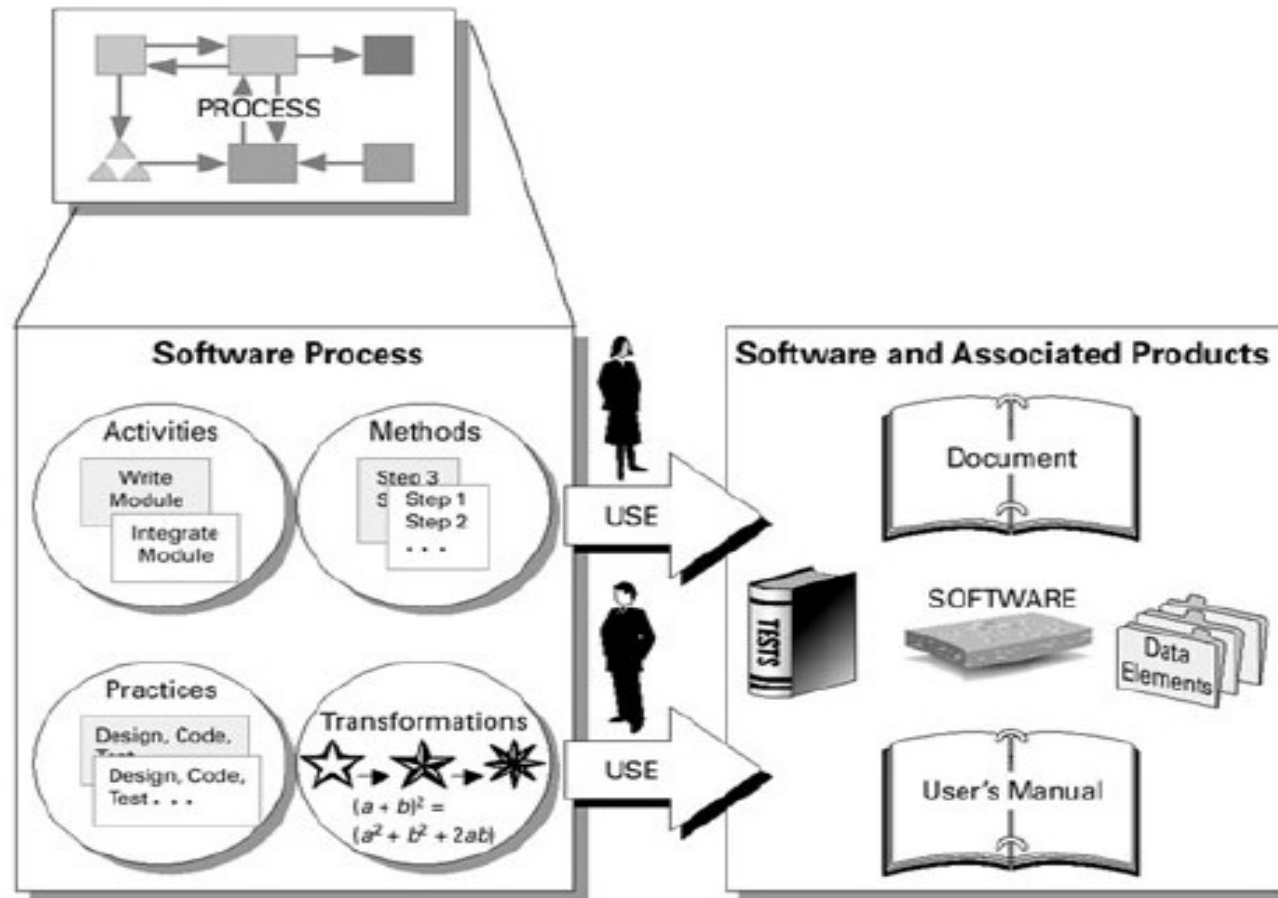
Ingineria software

- Ingineria software-ului este:
 - stabilirea si aplicarea principiilor ingineriei software pentru a obtine un sistem software de calitate in timpul si costul estimate,
 - aplicarea unei strategii sistematice, disciplinate si masurabile pentru dezvoltarea, utilizarea si intretinerea software-ului.
- Scop: dezvoltarea unui sistem software nou sau imbunatatirea unui existent



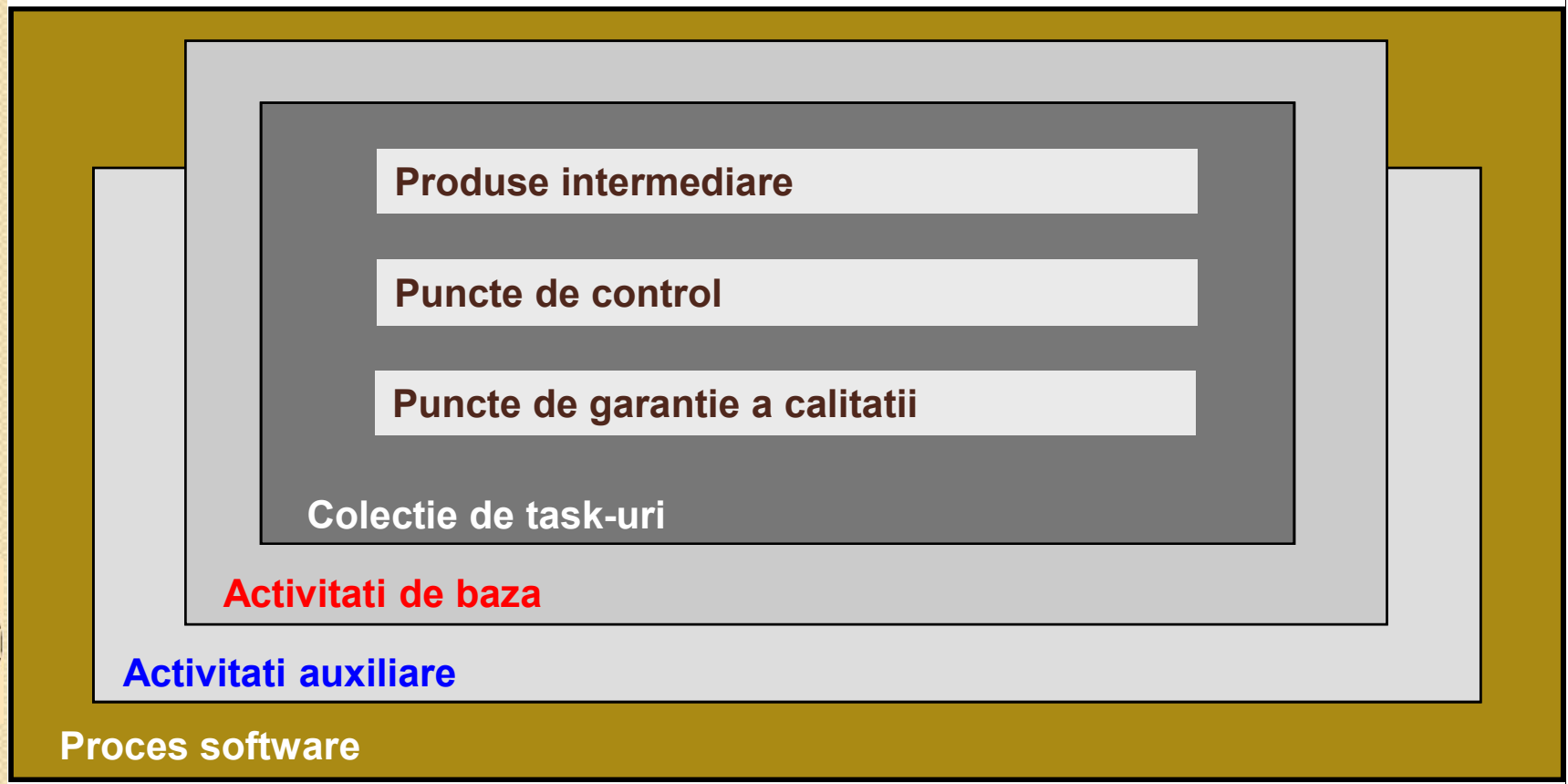
Proces software

- Proces care conduce la realizarea unui produs software
- O multime de activitati realizate de oameni si/sau masini cu scopul de a produce si intretine un sistem software si produse auxiliare: planul proiectului, documente de proiectare, cod, cazuri de testare, manual de utilizare, etc)



Proces software (cont.)

- Cadru de referinta in care se desfasoara toate activitatile necesare obtinerii unui sistem software de calitate
- Defineste strategia adoptata pentru producerea software-ului, nu si tehnologiile utilizate



Metodologii de dezvoltare a software

- **Metodologie** = colectie de metode folosite pentru rezolvarea unei clase de probleme, aplicabile in timpul dezvoltarii unui sistem si grupate intr-o anumita abordare generala, filozofica.
- O metodologie de dezvoltare sw consta din doua componente complementare:
 - Un proces software = secventa de activitati si decizii care muta sistemul de la concepere pana la terminare;
 - O notatie standard = forma de comunicare folosita pentru a documenta cunostintele si deciziile luate pe parcursul procesului de dezvoltare a sistemului.
- **Metodologia structurata** (1970-1989) se bazeaza pe principiile: abstractizare, ierarhizare, separarea datelor (entitatilor) de functii (procese).



Metodologii de dezvoltare a software (cont.)

- **Metodologia orientata spre date (1970-1980)** se bazeaza pe principiile:
 - datele intreprinderii furnizeaza o baza mult mai stabila pentru o proiectare de sistem decat procedurile intreprinderii, si
 - datele ar trebui vazute ca o resursa a intreprinderii independenta de sistemele care le proceseaza.
- Baza acestei metodologii a fost pusa in 1970 odata cu inventarea modelului relational de baze de date si a modelarii entitate-relatie.
- **Metodologia orientata spre obiecte (1990-prezent):** sistemul este modelat numai in termeni de obiecte. Aceste obiecte ofera servicii (comportament) lumii inconjuratoare si contin informatie. Obiectele comunica intre ele prin mesaje.



Principiile metodologiei orientata spre obiecte

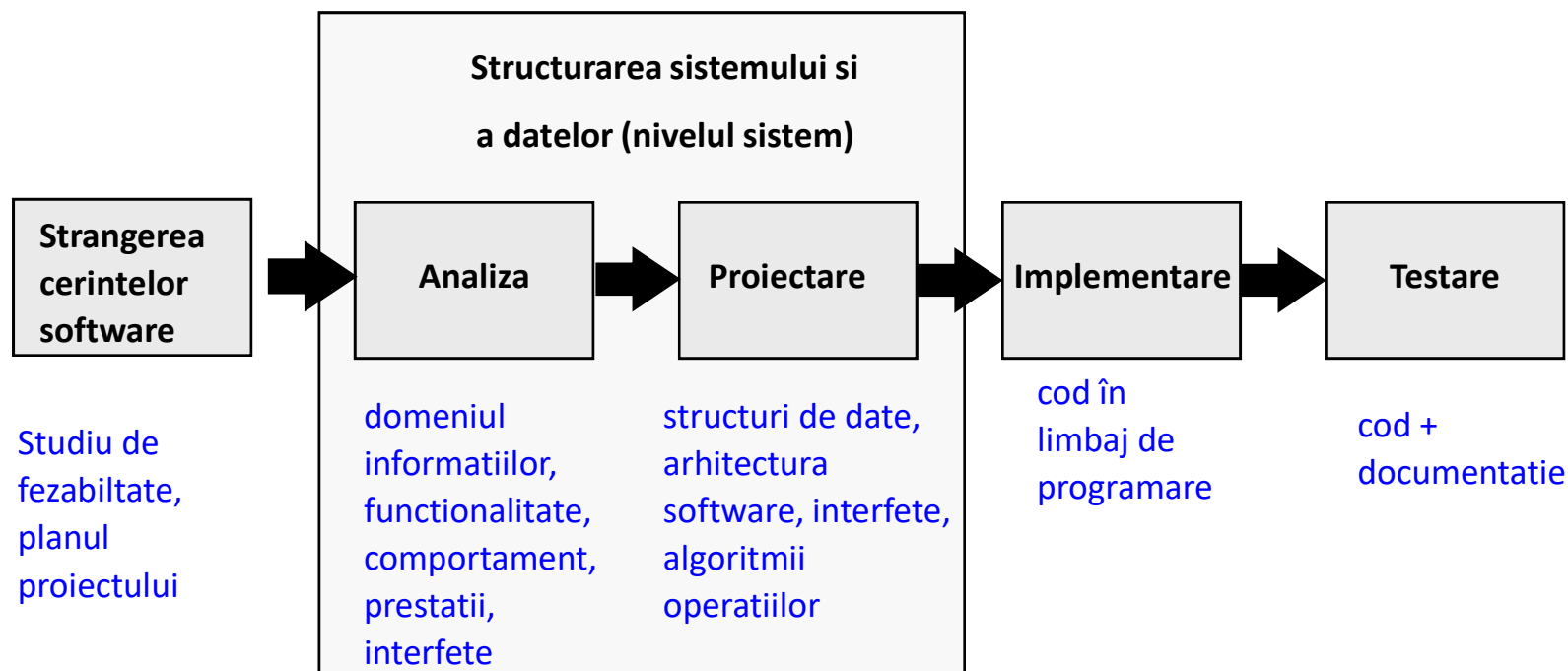
- **Abstractizare:** capacitatea de a capta proprietatile esentiale ale unei entitati, in cazul nostru, ale unui obiect.
- **Incapsulare:** datele (starea) si comportamentul (operatii) sunt incapsulate in obiect. Accesul la un obiect se face numai prin intermediul *interfetei* acestuia.
- **Mostenire:** mecanism folosit pentru partajarea datelor si a comportamentului comun unei colectii de clase. Aceasta factorizeaza proprietatile comune ale claselor in suprac clase si le reutilizeaza in definirea subclaselor.
- **Polimorfism:**
 - de incluziune: subclasificare
 - parametrizat: clase template
- **Modularitate:** Descompunerea unui sistem într-un ansamblu de module cu o mare coeziune interna si cu legaturi slabe intre ele.



Partea a II-a. Modele ale procesului software



Modelul in cascada

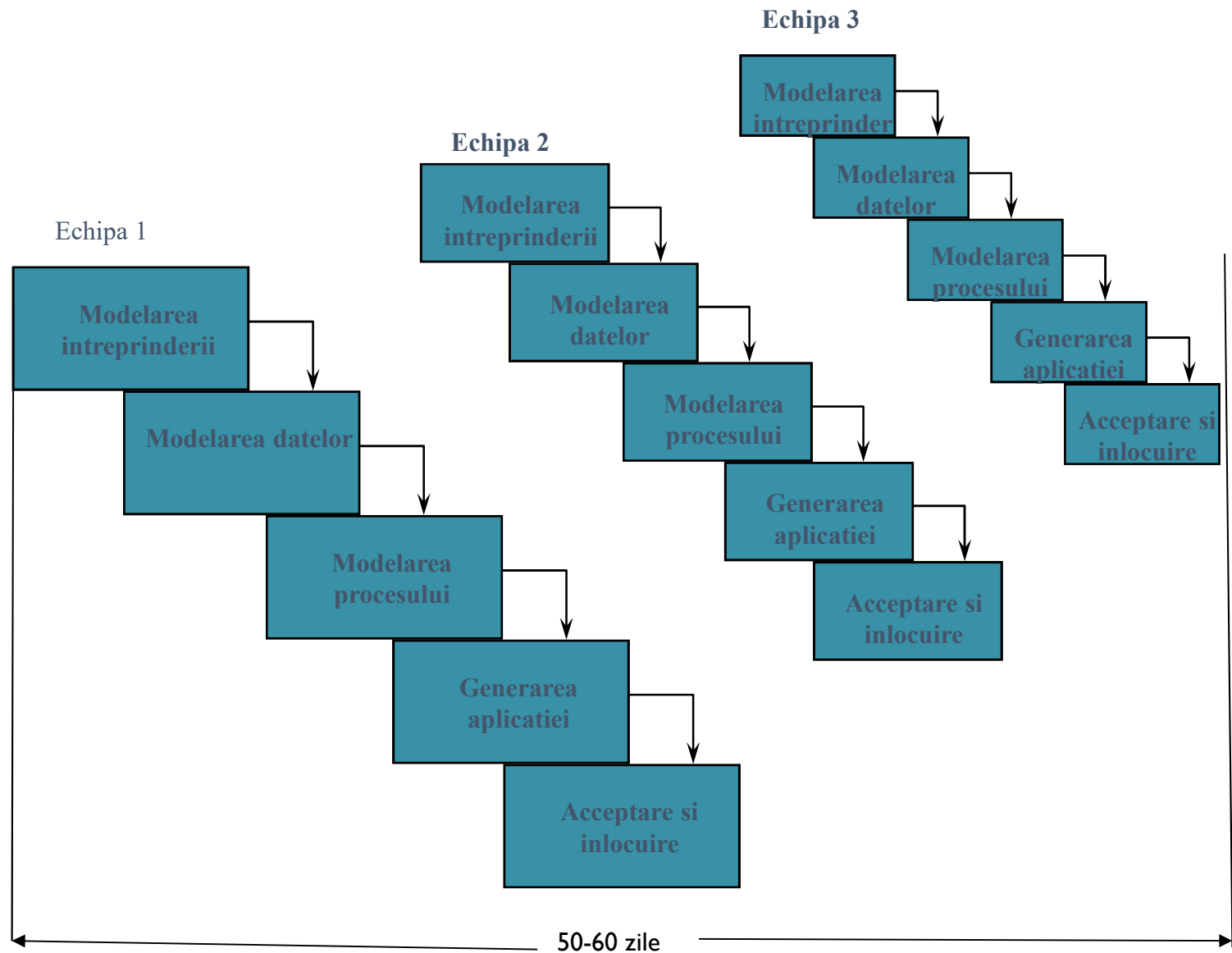


Probleme:

- proiectele reale nu respecta schema secventiala
- orice modificare in faza de proiectare determina modificarea cerintelor
- cerintele incomplete nu pot fi guvernate
- greselile logice banale sunt descoperite la inceputul perioadei de testare, in timp ce cele mai serioase sunt gasite ultimele
- versiune care sa functioneze se poate obtine doar la sfarsitul proiectului
- apar “stari de blocaj” datorate necesitatii de sincronizare intre activitati sau intre membrii echipei



Modelul RAD



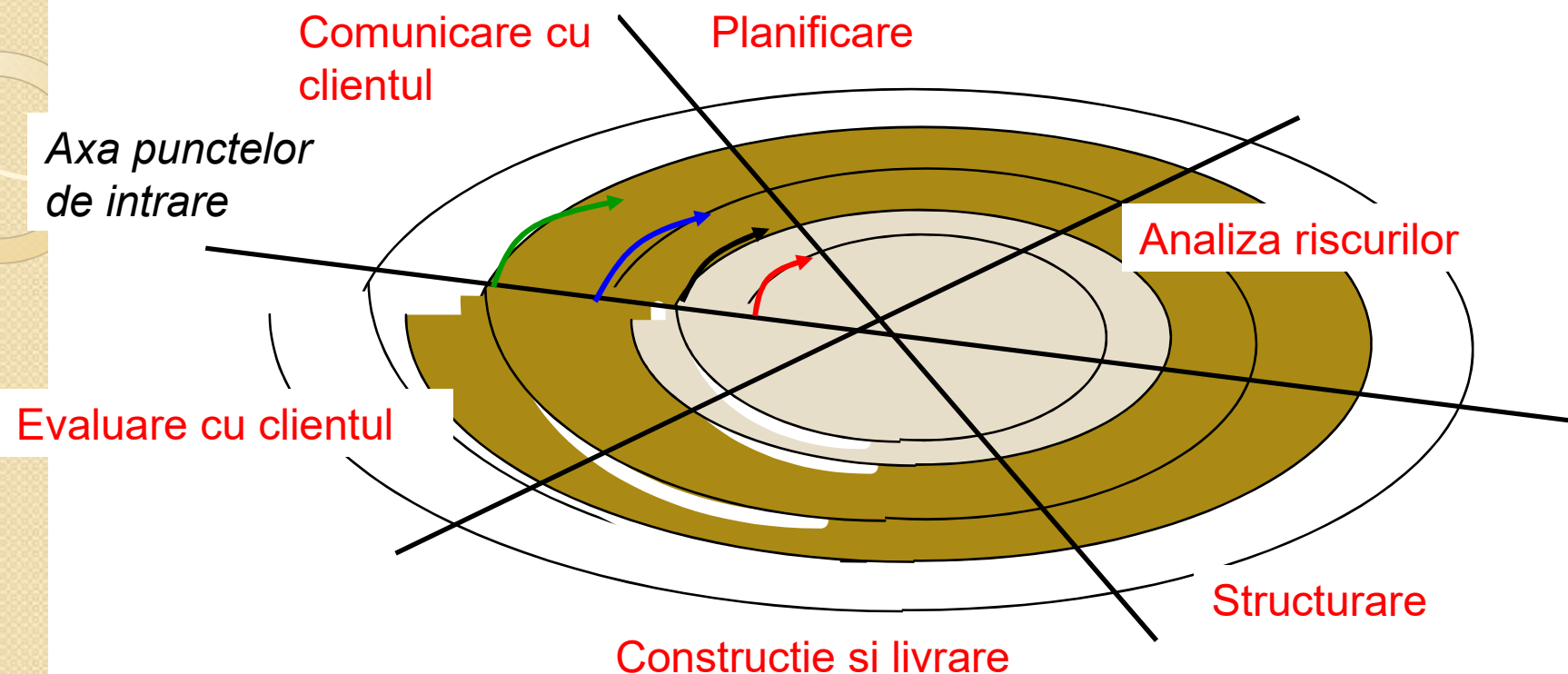
Modelul RAD (cont.)

- **Probleme:**

- Necesita numeroase resurse umane atunci cand proiectul este de dimensiuni mari.
- Daca angajamentul clientilor sau dezvoltatorilor de a realiza sistemul intr-un timp scurt nu se concretizeaza in fapte, proiectul esueaza.
- Nu functioneaza pentru sisteme care nu pot fi descompuse usor.
- Nu reuseste intotdeauna sa realizeze sisteme cu performante deosebite.
- Nu trebuie utilizat atunci cand noua aplicatie exploateaza tehnologii noi sau prevede un inalt grad de interoperabilitate cu programele existente.



Modelul in spirala



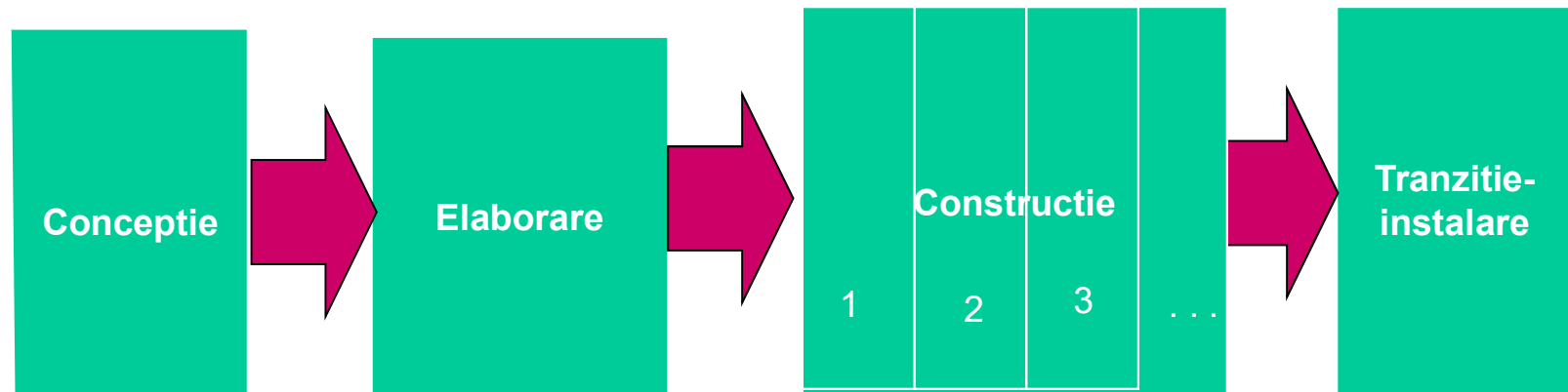
- **Probleme:**

- Este dificil sa fie convinsi clientii ca o dezvoltare de acest fel poate fi tinuta sub control;
- Necesita competente de nivel inalt pentru estimarea riscurilor;
- Daca un risc important nu este descoperit si controlat la timp, problemele pot creste ulterior exponential.



Modelul RUP

- Modelul procesului software RUP este format din 2 etape:
 - **etapa inginereasca**, condusa de echipe mai mici, dar mai putin predictibile, care realizeaza activitatile de analiza, proiectare si planificare
 - **etapa de productie**, condusa de echipe mai mari, dar mai mult predictibile, care realizeaza activitatile de implementare, testare si repartitie (deployment).



Faza de conceptie

- **Obiective generale:**

- Stabileste granitele sistemului si a conditiilor pe frontiera
- Identifica impreuna cu clientii si experti in domeniu cazurile de utilizare critice si scenariile fundamentale
- Propune una sau mai multe solutii pentru arhitectura sistemului
- Planifica dezvoltarea (estimeaza costurile si termenele intregului proiect), planifica in detaliu faza de elaborare
- Identifica si grupeaza riscurile

- **Activitati principale:**

- Formularea scopului proiectului
- Sintetizarea arhitecturii
- Planificarea si pregatirea unui business case



Faza de elaborare

- **Obiective generale:**

- Stabileste si documenteaza “vederea” externa asupra sistemului
- Stabileste si documenteaza arhitectura sistemului
- Planifica in detaliu faza de constructie
- Identifica componente ce se pot utiliza in faza de constructie si care ar putea influenta pozitiv asupra predarii

- **Activitati principale:**

- Elaborarea vederii externe a sistemului
- Elaborarea arhitecturii sistemului si selectarea componentelor
- Planificarea in detaliu a fazei de constructie



Faza de constructie

- **Obiective generale:**
 - Detaliaza arhitectura purtand-o spre implementare
 - Realizeaza iteratii ghidate de riscuri si de cerintele problemei, iteratii controlate de criterii de evaluare reexamine in mod regulat
 - Integreaza continuu
- **Activitati principale** deriva din indeplinirea obiectivelor principale
- **Criterii de evaluare:**
 - Aceasta versiune a sistemului este suficient de matura pentru a putea fi data utilizatorilor?
 - Aceasta versiune a sistemului este suficient de stabila pentru a fi data utilizatorilor?
 - Toti stakeholder-ii sunt pregatiti pentru instalarea produsului la utilizatori?
 - Sunt acceptabile cheltuielile actuale pe resurse vs cheltuielile planificate?



Faza de tranzitie-instalare

- **Obiective generale:**
 - faciliteaza acceptarea clientului (masoara gradul de satisfactie a clientului), imbuntateste prestatile, elimina erorile
 - porting al bazei de date existente in cea noua, eliminarea graduala a partilor mostenite
 - pregatirea personalului de operare si intretinere.
- **Activitati principale** deriva din indeplinirea obiectivelor principale
- **Criterii de evaluare:**
 - Clientul este satisfacut?
 - Sunt acceptabile cheltuielile actuale pe resurse vs cheltuielile planificate?
- Pentru unele proiecte, acest punct final al ciclului de viata poate coincide cu punctul de inceput al unui ciclu de viata pentru urmatoarea versiune a produsului.

